for Game Creator 2

Physics Car · Physics Bike · Hover Vehicle · Damage & Deformation · VFX · Audio · Entry / Exit · New Input System

RVR Gaming

Contents

About this asset

Rapid Vehicles is a drop-in vehicle kit for **Game Creator 2**. It ships three independent, physics-driven vehicle controllers — a **Physics Car**, a **Physics Bike**, and a **Hover Vehicle** — together with a set of supporting components that handle entering and exiting the vehicle, engine and drift audio, damage with mesh deformation, lights, and particle effects.

Every part of the kit is an ordinary Unity component: you add it in the Inspector and tune it with sliders, curves and toggles. Vehicles are turned on and off, and report their speed, RPM, health and fuel, through GC2 **Instructions** and **Properties**, so you can wire the kit into your game — dashboards, fuel pickups, damage states, mission logic — without writing any code.

This manual covers one-time project setup, a field-by-field reference for every component, the new Input System support, and a short troubleshooting section. Wherever a value is shown in `this font`, it is the exact field or method name as it appears in code or the Inspector.

> **TIP** If you only want a car driving in your scene as fast as possible, jump to **Quick Setup**, then **Setting up a drivable vehicle** near the end. The long component references are there for when you want to tune feel.

Requirements

- **Game Creator 2** — installed and working in your project. The vehicle controllers expose GC2 Property fields and Instruction hooks.
- **Shader Graph** — install from **Window** → **Package Manager**. Used by the kit's materials, the glow on the lights, and the hover effects.
- **Visual Effect Graph** — install from **Window** → **Package Manager**. Required for the damage, exhaust and drift-smoke effects in `VehicleVFX`.
- **TextMeshPro** — imported (TMP Essentials) if you use the included UI. Add an EventSystem to the scene if UI input does not respond.
- **Input System package** — optional. Only required if you want to drive with the new Input System — see **Input**. The kit compiles and runs on the legacy Input Manager without it.

Quick Setup

Three short steps get a vehicle running in a fresh project. Do them once, in order.

First-time setup

1. **Add a Terrain layer.** In **Project Settings** → **Tags and Layers**, add a layer named `Terrain`. Assign it to your ground/terrain and anything the hover vehicle should float over.
2. **Assign the Terrain mask.** On the **Hover Vehicle Controller**, open the **Terrain** section and set `TerrainMask` to that layer. (The car and bike use Wheel Colliders and don't need this.)
3. **Install the render packages.** In the **Package Manager**, confirm **Shader Graph** and **Visual Effect Graph** are installed. If UI misbehaves, confirm **TextMeshPro** and an **EventSystem** are present.

> **NOTE** The hover vehicle aligns itself to whatever is on the `Terrain` layer. If it falls through the world or floats at the wrong height, the layer or the `TerrainMask` is almost always the cause.

Quick overview video: https://youtu.be/kCwbW5YP1Cw

Quick setup video: https://youtu.be/GG-XleRpLE0

Core concepts

One controller per vehicle

A vehicle has exactly one of the three controllers on its root object: `PhysicsCarController`, `PhysicsBikeController`, or `HoverVehicleController`. The supporting components (audio, VFX, entry, lights, deformation) detect which controller is present and adapt automatically, so the same effect scripts work on all three vehicle types.

Enabling and disabling

Each controller has an `isVehicleEnabled` flag shown as the **Vehicle Enabled** toggle in the header of its Inspector. While disabled, the vehicle ignores input and idles. The Entry components flip this for you when a character gets in or out, but you can also drive it yourself from GC2 (see each controller's **Methods you can call**).

Reading speed, RPM, health and fuel

The controllers continuously write their live state into GC2 **Set Number Property** fields — `currentSpeed` (km/h), `currentRPM`, `currentHealth` and `currentFuel`. Point these at GC2 variables and you can drive speedometers, fuel gauges, damage logic or mission triggers entirely with Instructions and Conditions.

> **TIP** Set Number Property fields are outputs, not inputs. To **change** health or fuel (refuel pickups, damage zones) write to the same GC2 variable they target; the controller reads its starting values from the linked properties on play.

The Physics Car Controller

Add `PhysicsCarController` to the car's root (it needs a `Rigidbody` and four `WheelColliders`). The Inspector is grouped into foldouts.

Base Stats

- **topSpeed** — maximum forward speed, in km/h.
- **currentSpeed** — GC2 output: the live speed in km/h. Link to a variable for a speedometer.
- **acceleration** — how hard the car pulls toward top speed.
- **braking** — braking strength when reversing input is applied while moving forward.

Advanced Physics

- **reverseSpeed / reverseAcceleration** — top speed and pull when driving in reverse.
- **accelerationCurve / accelerationCurveCoefficient** — shapes how acceleration tapers as speed climbs, so the car feels strong off the line and softer near top speed.
- **coastingDrag** — how quickly the car slows when there is no throttle.
- **grip** — lateral tyre grip — higher resists sliding, lower lets the back step out.
- **steer** — steering strength / responsiveness.
- **handbrakeForce** — extra rotational slide applied while the handbrake is held.
- **addedGravity** — downward force added on top of gravity for a planted feel and less air-time.

Transmission

- **numberOfGears** — how many gears the simulated gearbox steps through.
- **maxRPM / idleRPM** — engine RPM ceiling and idle floor; drive the audio pitch and the gauge.
- **currentRPM** — GC2 output: live engine RPM.
- **gearRatio** — multiplier mapping wheel speed to RPM per gear.

Drifting

- **driftGrip** — reduced grip used while drifting, controlling how loose the slide is.
- **driftControl** — how much steering authority you keep mid-drift.
- **driftDampening** — how quickly the slide settles back to normal grip.

Damage & Fuel

- **maxDamage / currentHealth** — damage ceiling and the GC2 health output. At zero health the exhaust stops and the damage VFX plays.
- **collisionThreshold** — minimum impact magnitude before a collision counts as damage.
- **damageIntensity** — how much a qualifying impact reduces health.

- **fuelCapacity / currentFuel / fuelEfficiency** — tank size, the GC2 fuel output, and how fast fuel is consumed under load. At zero fuel the engine cuts out.

Wheels & Suspension

Assign the four wheel **meshes** (`frontLeftWheelTransform` … `rearRightWheelTransform`) and the four `WheelColliders`. Tune ride with `suspensionHeight`, `suspensionSpring` and `suspensionDamp`.

> **TIP** Use the **Auto Setup Wheel Colliders** button at the bottom of the Inspector to create and align the four colliders from your wheel meshes, then fine-tune suspension by hand.

Center of Mass & Visual Components

- **centerOfMass** — local offset for the Rigidbody centre of mass — lower it to reduce roll-over.
- **carBody** — the visual body transform (also used as the parent for the entry seat).
- **steeringWheelMesh / steeringWheelMaxAngle / steeringWheelRotationAxis** — optional in-cabin steering wheel that rotates with steering input, around the chosen local axis.

Methods you can call

- **SetCarEnabled(bool)** — enable or disable the car (used by Car Entry).
- **SetDriftInput(bool)** — begin/end a drift — bind to a GC2 button Instruction.
- **SetHandbrakeInput(bool)** — hold/release the handbrake from GC2.
- **ResetVehicle()** — restore the car to a clean state (health, orientation, motion).
- **AutoSetupWheelColliders()** — editor helper described above; also callable at runtime.

The Physics Bike Controller

Add `PhysicsBikeController` to the bike's root. It shares the car's Base Stats, Advanced Physics, Transmission, Drifting, Damage and Fuel groups (same field names and meaning), plus lean and stability tuning unique to a two-wheeler.

Leaning & stability

- **leanTorque** — how strongly the bike leans into a turn.
- **maxLeanAngle** — maximum lean from vertical.
- **leanSpeedThreshold** — speed above which leaning becomes active.
- **driftLeanMultiplier** — extra lean applied while drifting (the Lean Tweaks group).
- **airAngularDamping** — angular damping while airborne, to stop the bike spinning wildly off jumps.
- **driftAngularDamping** — angular damping during a drift, controlling how twitchy the slide is.

Wheels & Visual Components

Two wheels here: `frontWheelTransform` / `rearWheelTransform` and `frontWheelCollider` / `rearWheelCollider`. Visuals use `bikeBody`, an optional `steeringWheelMesh` (handlebars) with `steeringWheelMaxAngle`, and `frontWheelMaxAngle` for the steered front wheel.

Methods you can call

- **SetBikeEnabled(bool)** — enable or disable the bike (used by Bike Entry, via reflection).
- **SetHandbrakeInput(bool)** — hold/release the handbrake from GC2.
- **ResetVehicle() / AutoSetupWheelColliders()** — as on the car.

The Hover Vehicle Controller

Add `HoverVehicleController` to the hover vehicle's root. Instead of wheels it floats above the `Terrain` layer using a set of hover points, so its tuning is force-based rather than wheel-based.

Base Stats

- **MoveForce / ReversingForce / TurnForce / BoostForce** — forward, reverse, turning and boost thrust.
- **HoverMultiplierInMotion / HoverMultiplierAtRest / HoverExponent** — how strongly the craft is pushed up while moving versus stationary, and the falloff curve of the hover force with height.
- **currentSpeed** — GC2 speed output.

Boost Resource

- **maxBoost / currentBoost / boostEfficiency** — boost tank size, the GC2 output, and consumption rate.
- **BoostCooldown** — delay before boost can be used again after depletion.
- **AirControlMultiplier** — how much steering authority you keep while airborne.

Slope, Landing & Drag

- **MaxDownhillForce / MaxDownhillAngle** — downhill assistance and the angle it applies over.
- **UpthrustAngleLimit / SlopeAngleLimit** — limits on how steep a slope the craft will climb or hug.
- **GroundedThreshold** — height under which the craft counts as grounded.
- **LandingDownforceDuration / DescentSlowMultiplier / DescentSlowExponent / DescentSlowSpeedMultiplier** — shape the settle on landing so it doesn't bounce.
- **DragWhenGrounded / DragInAir** — drag applied near the ground versus in the air.

Rotational Forces & Throttle

- **ZRebalanceThreshold / ZRebalanceForce / XRebalanceThreshold / XRebalanceForce** — self-levelling that returns the craft to flat when it tilts past a threshold on each axis.
- **MaxBankAngle / BankSpeed** — how far and how fast it banks into turns.
- **ThrottleAcceleration / ThrottleMaxSpeed / BoostThrottleAcceleration / BoostThrottleMaxSpeed** — throttle ramp and speed caps in normal and boosting states.

References, Terrain & Visuals

- **hoverPoints** — the transforms a hover force is applied at — typically four, near the corners. More points = more stable.
- **TerrainMask** — the layer(s) treated as ground for hovering and alignment.

- **modelTransform** — the visual model that banks and tilts.

Methods you can call

- **SetBoostInput(bool)** — begin/end boosting — bind to a GC2 button Instruction.

> **NOTE** On the hover vehicle, `isVehicleEnabled` defaults to `true` so it can idle and hover in a scene before a driver gets in. The car and bike default to `false`.

Input

All three controllers can read driving input from Unity's **legacy Input Manager**, the new **Input System**, or both at once. The **Input** foldout sits at the bottom of every vehicle's Inspector.

Fields

- **Input Mode** — a dropdown with three choices. **Legacy** reads the old Input Manager (default, matches existing projects). **New** reads the Input System asset below. **Both** reads either, so a keyboard and a gamepad can be used interchangeably.
- **Input Action Asset** — only shown when the Input System package is installed. Drop your `.inputactions` asset here.
- **Move Action** — a dropdown listing every action in that asset, grouped by action map (for example `Player/Move`). Pick the **Vector2** action that should drive the vehicle.

How the Move action is read

The selected action is read as a Vector2 each frame and mapped to the controls:

- **Car & Bike:** `x` = steer (left/right), `y` = throttle / brake (forward/back).
- **Hover:** `x` = strafe / turn, `y` = forward / back.

A standard `Move` action from a default Input Actions asset (WASD + left stick, Vector2) already fits. In **Both** mode the legacy and new values are summed and clamped, so either device works.

Setting it up

1. Install **Input System** from the Package Manager (Unity may prompt to enable it and restart).
2. Set **Input Mode** to **New** (or **Both**) on the vehicle.
3. Assign your Input Action Asset, then choose the **Move** action from the dropdown.

> **NOTE** If you set Unity's **Active Input Handling** (Player Settings) to **Input System Package (New)** only, the legacy path is compiled out — use **New** mode. With **Both** selected there, every Input Mode works.
>
> **TIP** Drift, handbrake and boost are driven by GC2 Instructions (`SetDriftInput`, `SetHandbrakeInput`, `SetBoostInput`), not by the Move action — bind those to whatever buttons you like in your own actions asset or GC2 triggers.

Vehicle Entry & Exit

Two components handle a GC2 character getting in and out: `CarEntry` (car or hover) and `BikeEntry` (bike or hover). They play an animation, parent the character to a seat, hand the controls over, and run your GC2 Instructions on enter and exit. Call `EnterCar` / `ExitCar` (or `EnterBike` / `ExitBike`) from a GC2 Trigger, passing the Character.

Car Entry — key fields

- **entryAnimation / exitAnimation / animationMask** — clips and mask played on the character; transition times and useRootMotion control the blend.
- **doorTransform / doorOpenRotation / door…Delay / doorRotationDuration** — optional door that swings open on entry and closes after a delay.
- **drivingState / drivingStateLayer / transitions** — a GC2 State applied while seated (driving pose).
- **entryParent** — the seat transform; it is re-parented under the car body so it moves with the vehicle.
- **steeringWheel…HandTarget / …IKWeight** — hand IK targets so the driver holds the wheel.
- **onEnter / onExit** — GC2 Instruction lists run when entry / exit completes.

Bike Entry — additions

Bike Entry adds foot IK and a hover-lift. Foot targets (`leftFootTarget`, `rightFootTarget`) keep feet on the pegs; `groundLeftFootTarget` plants the left foot down when stopped, blended over `footSmoothingTime`. When attached to a hover vehicle it gently lifts the craft on mount. It enables the bike via `SetBikeEnabled`.

> **NOTE** Both Entry components also work on the **Hover Vehicle**: Car Entry suits a cockpit-style craft, Bike Entry suits a sit-on speeder bike (with the lift-on-mount effect).

Effects & Feedback

Vehicle VFX

Add `VehicleVFX` to the vehicle root. It auto-detects the controller and drives four effects. Skid marks and drift smoke are wheel-based and are skipped on the hover vehicle.

- **Damage VFX** — a VFX Graph prefab spawned at `damageVFXOffset/Rotation`; plays when health hits zero, stops a few seconds after repair.
- **Exhaust VFX** — plays while the vehicle is enabled and has health and fuel; stops otherwise.
- **Skid Marks** — LineRenderer trails laid down while drifting or handbraking, using `skidMarkMaterial`, fading over `skidFadeTime`.
- **Drift Smoke** — a pooled smoke prefab (`driftSmokePoolSize`, `driftSmokeLifetime`) emitted from the wheels mid-slide.

Vehicle Lights

Add `VehicleLights` for headlights and tail-lights. Assign the front/back light **mesh renderers** and optional `spotLight1/2`.

- **frontLightOnColor / backLightOnColor** — HDR emissive colours written to the meshes' `_GlowColor` when lit.
- **spotLightIntensity** — intensity the spot lights jump to when on.
- **LightsOn() / LightsOff()** — call from GC2 to toggle; `AreLightsOn` reports state.

> **NOTE** The mesh glow uses a material property named `_GlowColor`. Your headlight material needs that property (the kit's light shader provides it) for the meshes to light up.

Vehicle Audio

Add `VehicleAudioPhysics` (on the vehicle or a child) for engine, drift and collision sound. It reads the car or bike controller in its parents.

- **Engine** — pitch and volume lerp between idle and max as `currentRPM` moves between `idleRPM` and `maxRPM`; `smoothFactor` softens the change.
- **Drift** — a looping drift clip whose volume rises while drifting or handbraking.
- **Collision** — a one-shot played when an impact exceeds `collisionThreshold`.

Vehicle Deformation

Add `VehicleDeformation` to dent body panels on impact. List the `MeshFilter`s to deform under **Meshes to Deform**.

- **deformationRadius / deformationIntensity / maxDeformation** — how far each hit spreads, how deep it pushes, and the per-vertex cap.
- **recoverySpeed** — optional rate at which dents ease back out over time.

- **deformationCooldown** — minimum gap between deformation passes, to avoid per-frame spikes.
- **ApplyDeformation(point, damage, dir) / ResetDeformation()** — dent at a world point, or pop all panels back to factory.

Setting up a drivable vehicle

A minimal car, from an empty object:

1. Create the car: a root with a `Rigidbody`, a body mesh, four wheel meshes and a `BoxCollider`.
2. Add `PhysicsCarController`. Assign the wheel **meshes**, then click **Auto Setup Wheel Colliders**.
3. Link the GC2 outputs — `currentSpeed`, `currentRPM`, `currentHealth`, `currentFuel` — to variables if you want a HUD.
4. Set **Input Mode** (Legacy is fine to start), or assign your Input Action Asset and Move action for the new system.
5. Add `CarEntry` and call `EnterCar` from a GC2 “press to interact” Trigger. Set `entryParent` to a seat transform.
6. Optionally add `VehicleAudioPhysics`, `VehicleVFX`, `VehicleLights` and `VehicleDeformation` and assign their references.
7. Press Play, enter the car, and drive.

Troubleshooting

The vehicle won't move

Check **Vehicle Enabled** is on (Entry components set this on enter), that there is fuel and health, and — if using the Input System — that **Input Mode** is New/Both and a **Move** action is selected.

New Input mode does nothing

Confirm the Input System package is installed, the `InputActionAsset` is assigned, and the chosen Move action is a **Vector2**. If **Active Input Handling** is set to New-only, the legacy path is off — use New or Both.

Legacy input throws an error at runtime

That happens when **Active Input Handling** is set to Input System only while a Legacy/Both mode tries to read the old manager. Switch Active Input Handling to **Both**, or set the vehicle's Input Mode to New.

The hover vehicle falls through the floor

The ground isn't on the `Terrain` layer, or `TerrainMask` doesn't include it. Fix the layer and the mask.

Wheels spin but the car doesn't grip

Run **Auto Setup Wheel Colliders**, lower the `centerOfMass`, and check the Wheel Colliders actually touch the ground.

Headlights don't glow

The light material needs a `_GlowColor` property, and the colours should be HDR (intensity > 1). Spot lights need to be assigned to `spotLight1/2`.

Materials are pink or VFX don't render

Install **Shader Graph** and **Visual Effect Graph** from the Package Manager and let the project reimport.

No skid marks or drift smoke

These are wheel-based and are disabled on the hover vehicle by design. On the car/bike, assign `skidMarkMaterial` and a `driftSmokePrefab`.

Support

- **Video tutorials for GC2** — the RVR Gaming YouTube channel.
- **Asset support** — the RVR Gaming Discord.

Rapid Vehicles — RVR Gaming